

Practical Exercise: Deadlocks

1. Consider the following system snapshot using the data structures in the Banker's Algorithm with resources A, B, C, D and processes P1, P2, P3, P4 and P5:

	Maximum				Allocated				Required				Available			
	A	B	C	D	A	B	C	D	A	B	C	D	A	B	C	D
P ₁	12	0	1	2	4	0	0	1								
P ₂	2	7	5	0	1	3	0	0								
P ₃	4	3	5	5	1	2	5	4								
P ₄	6	6	6	3	0	6	3	3								
P ₅	5	6	5	6	0	2	1	2								
													20	2	0	5

Using the Banker's algorithm answer the following:

- How many resources of type A, B, C and D are there?
 - What are the contents of the "Required" matrix?
 - Is the system in a safe state? If so, show the order in which the processes can be allocated the requested resources?
2. Suppose that a system is in an unsafe state. Show that it is possible for the processes to complete their execution without entering a deadlock state.
3. Consider the following resource-allocation policy. Requests for and releases of resources are allowed at any time. If a request for resources cannot be satisfied because the resources are not available, then we check any processes that are blocked waiting for resources. If a blocked process has the desired resources, then these resources are taken away from it and are given to the requesting process. The vector of resources for which the blocked process is waiting is increased to include the resources that were taken away.

For example, consider a system with three resource types and the vector Available initialized to (4,2,2). If process P0 asks for (2,2,1), it gets them. If P1 asks for (1,0,1), it gets them. Then, if P0 asks for (0,0,1), it is blocked (resource not available). If P2 now asks for (2,0,0), it gets the available one (1,0,0) and one that was allocated to P0 (since P0 is blocked). P0's Allocation vector goes down to (1,2,1), and its Need vector goes up to (1,0,1).

- Can deadlock occur? If you answer "yes," give an example. If you answer "no," specify which necessary condition cannot occur.
- Can indefinite blocking occur? Explain your answer.

Practical Exercise: Memory Management

Consider the following page reference string:

1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6.

How many page faults would occur for the following replacement algorithms, assuming one, two, three, four, five, six, or seven frames? Remember all frames are initially empty, so your first unique pages will all cost one fault each.

- Optimal replacement
- LRU replacement
- FIFO replacement